InvenSight

From inventory liquidation to full demand intelligence

Dual-sided retail platform: AI markdown + redistribution + demand forecasting

Extended Hackathon Strategy & Technical Playbook -- Volume II

Continues from Volume I, Phase 7 · Companion file: RULE.md

InvenSight

From inventory liquidation to full demand intelligence

A dual-sided retail platform: AI markdown + redistribution + demand forecasting

Extended Hackathon Strategy & Technical Playbook — Volume II

This document extends the original InvenSight playbook. It does not repeat what was already covered — it picks up from the working Phase 0/1 build you already have, and lays out how to layer demand forecasting and inventory optimization on top of it without throwing away what's built. Companion file: `RULE.md`, provided separately.

Chapter 1 — The problem, restated with the forecasting angle

1.1 The two problems are actually one problem, seen from two ends

The original playbook covered the **liquidation** side: what happens to inventory that is already at risk of expiring. This volume covers the other half of the same failure — **stores don't know how much to order in the first place**, so they over-stock (creating the waste you're rescuing) and under-stock (creating empty shelves and lost sales) in the same week, often for the same SKU.

- Independent research consistently finds that **over-ordering driven by poor demand visibility is the single largest upstream cause of retail perishable waste** — more than shelf-life mismanagement or slow markdown response. Fixing markdown speed (your existing engine) treats the symptom; forecasting treats the cause.
- The inverse failure — under-ordering — costs stores in a way that never shows up in a waste report: a stockout is invisible in most retail analytics, it just looks like "no sale happened."

1.2 Why forecasting is hard for a typical store to do well

- **Manual, spreadsheet-based reordering** is still the norm outside large chains — a store manager eyeballing "how much did we sell last week" with no seasonality, weather, or promotion adjustment.
- **No unified view across SKUs** — a manager can hold maybe a dozen products' trends in their head; a mid-size grocery store carries thousands.
- **Reactive, not predictive** — most reordering happens *after* a shelf is already empty or already overflowing, not before.
- **Data exists but is siloed** — POS systems log every sale, but that data rarely feeds back into ordering decisions in a systematic way.

1.3 Who suffers, and what they now expect (updated)

Stakeholder	Original expectation (Volume I)	Added expectation (this volume)
Grocery / retail stores	Automated markdown, less manual labor	Tell me **how much to order next week**, not just what to discount today
Price-sensitive consumers	Real-time discount alerts nearby	(unchanged — still served by the existing claim flow)
Shelters / NGOs	Predictable surplus access	(unchanged)
Store category/purchasing managers *(new stakeholder surfaced by this pivot)*	—	A trend view per category, a reorder quantity recommendation, and a confidence level they can act on without a data science background

1.4 The framing that makes this pitch stronger, not just bigger

Judges see a lot of "waste reduction" pitches and a lot of "demand forecasting" pitches — usually as two separate products from two separate teams. The reason to combine them under InvenSight rather than bolt on a second unrelated feature: **the same batch-level data your inventory system already tracks (sales velocity, expiry outcomes, claim-vs-expire ratios) is exactly the training signal a forecasting model needs.** You are not building a second product. You are pointing a second model at data your first system is already generating.

Chapter 2 — Our solution & market significance (updated)

2.1 What InvenSight becomes with this addition

InvenSight was a **reactive-to-proactive-pricing** engine (dynamic markdown + redistribution). With demand forecasting layered in, it becomes a **closed-loop inventory intelligence platform**:

```
FORECAST (how much to order) → STOCK (order placed) → MONITOR (Phase 1-3, already built)
→ MARKDOWN (Phase 2, already built) → RESCUE/SELL (Phase 3-5, already built)
→ OUTCOME DATA → feeds back into FORECAST
```

This is the structural pitch: **every other player does one arrow in that loop. InvenSight closes the whole loop.**

2.2 What's genuinely new vs. what's reused

Component	Status
`stores`, `batches` tables, ingestion, CRUD	**Reused as-is** — this is your Phase 1 work, unchanged
Markdown model (Model A), segment classifier (Model B)	**Reused as-is** — this volume adds a model, doesn't replace these
Geofencing, claims, comms, payments	**Reused as-is**
Eco-Impact dashboard	**Extended**, not replaced — a new tab, not a new dashboard
Demand forecasting model (Model D)	**New** — this volume's core addition
Reorder recommendation dashboard	**New** — the retailer-facing surface for Model D's output
Historical sales ingestion path	**New**, but reuses the CSV ingestion pattern from Phase 1

2.3 Market significance, restated with the forecasting angle

Retail demand forecasting and inventory optimization is its own large, independently-funded software category — separate from, but adjacent to, the food-waste-management category covered in Volume I. Vendors solving *only* this problem (see Chapter 3) have raised real institutional funding on the strength of "reduce over-ordering" alone, without a redistribution or markdown layer at all. InvenSight's pitch is that **combining forecasting with the liquidation/redistribution loop you've already built is strictly more defensible than either half alone** — a forecasting-only competitor still leaves a store with a shrinkage problem when the forecast is wrong; a markdown-only competitor (your Volume I positioning) still leaves a store over-ordering next week.

2.4 The two-sided value story, sharpened for the pitch

- **For the customer**: nothing changes about what they experience — they still get the discounted item via SMS/WhatsApp, no login, same claim flow.
- **For the retailer**: the pitch upgrades from *"we clean up your mistakes"* to *"we help you avoid the mistake in the first place, and clean up what's left."* This is a materially stronger sell to an actual retail buyer, because purchasing/reordering decisions are made by a different, more senior stakeholder than markdown decisions — and that's the person who controls a bigger budget line.

Chapter 3 — Competitive landscape, updated for the forecasting angle

3.1 A third lane, mapped alongside Volume I's table

Volume I mapped food-waste tech into three lanes (consumer marketplace, community sharing, pricing infrastructure) plus a forecasting/ordering lane occupied by Afresh and Shelf Engine, which were noted as *complementary, not competitive* — solving a different pipeline stage. This volume changes that relationship: by adding forecasting, **InvenSight now directly overlaps with that lane** and needs an honest answer for it.

Player	What they actually do	Where InvenSight now overlaps	Where InvenSight still differs
Afresh / Shelf Engine	AI demand forecasting to prevent over-ordering, upstream of markdown	Directly — this is now the same category as your Model D	They stop at the order; they do not touch markdown, redistribution, or the consumer-facing claim flow
Wasteless	AI dynamic pricing on the shelf-edge	Already covered in Volume I	Still no forecasting layer, still no consumer alert layer
Too Good To Go / Flashfood	Consumer marketplace for discounted surplus	Already covered in Volume I	Neither has any ordering/forecasting layer at all

3.2 The honest caveat

Afresh and Shelf Engine are mature, funded products with real retailer deployments and multi-year data advantages — a hackathon-built forecasting model will not out-predict a system trained on years of a specific chain's real POS history. The honest pitch angle is **not** "our forecasts are more accurate." It is: **"we are the only system where a forecast, a markdown, and a redistribution channel all read from and write to the same batch-level data — nobody else closes that loop."** Accuracy-of-forecast is a red herring to defend against in Q&A; structural integration is the actual differentiator, and it's a true one.

3.3 What a judge will likely push on, and the honest answer

- *"Isn't this just Afresh with extra steps?"* — No: Afresh's output (a recommended order) is where its product ends. InvenSight's forecast output feeds the same batch record that your markdown engine already watches — so an item that arrives under-selling relative to forecast gets flagged for earlier markdown, automatically, without a human connecting the two systems. That connective tissue is the product.
- *"Your forecast is trained on synthetic/small data — how good can it really be?"* — Be upfront (see Chapter 12.5): state it's synthetic, explain why (no real retailer feed available for a hackathon), and frame the demo around the **pipeline being real**, not the specific numbers being production-grade.

Chapter 4 — Technical overview of the extended project

4.1 A fourth plane, added to the existing three

Volume I described the system as three cooperating planes (Data & Prediction, Orchestration, Delivery). This volume doesn't add a fourth plane — it **extends the Data & Prediction plane**, because forecasting is architecturally the same kind of work as markdown pricing: read historical data, run a model, write a recommendation back to a table.

Plane	Volume I responsibility	This volume's addition
Data & Prediction	Ingest batch data, run markdown model, decide price + segment	**+ Ingest historical sales, run forecasting model, decide reorder quantity + confidence**
Orchestration	Reservations, locking, payments, standing orders	Unchanged
Delivery	Claim page, Twilio, Stripe/Razorpay, dashboards	**+ New reorder-recommendation view inside the existing Store Dashboard**

4.2 The event loop, extended

Volume I's event loop started at "a scheduled scan finds at-risk batches." This volume adds a loop that runs *before* that one, at a different cadence:

```
WEEKLY/DAILY: historical sales + season/weather/event context
  → forecasting model → recommended reorder qty + confidence per SKU/store
  → surfaced on retailer dashboard (accept / override)
  → order placed (manually, outside InvenSight's scope for the hackathon)
  → new stock becomes a batch (Phase 1 ingestion, already built)
  → [Volume I's loop takes over from here: markdown → claim → sale/donation]
  → outcome (sold/expired/donated) feeds BOTH the markdown model's retrain
    AND the forecasting model's next cycle
```

The key design point: **the forecasting loop's output is not a new kind of record.** A reorder recommendation, once acted on, becomes exactly the same `batches` row your Phase 1 ingestion already handles. No new core entity is required — which is why this integrates cheaply into what you've built.

Chapter 5 — Architecture: client side & server side (extended)

5.1 Client side — one new surface, not a new app

- **Retailer Dashboard (existing, Phase 6)** gains a new tab: **Demand & Reorder**. Same React app, same auth, same layout shell — a new route and a new set of components, not a separate application.
  - Trend chart per SKU/category (actual sales vs. forecast, last 8-12 weeks)
  - Reorder recommendation table: SKU, recommended quantity, confidence band, “why” (top 2-3 contributing factors — e.g. “trending up 12% + upcoming local festival”)
  - Accept / adjust / dismiss controls per recommendation — this human-in-the-loop step matters both for trust and for the demo narrative
- **No new consumer-facing surface.** The claim page is untouched by this volume.

5.2 Server side — one new model, one new table, no new services

- **New table**: `sales_history` (see Chapter 11) — append-only, one row per SKU/store/day.
- **New table**: `reorder_recommendations` — one row per SKU/store/forecast run, holding the model’s output and the retailer’s accept/override decision.
- **New Celery task**: a scheduled forecast run (daily or weekly, not real-time like the markdown scan) — reuses the existing Celery beat infrastructure from Phase 0/2, just a second scheduled job alongside the expiry-risk scan.
- **New FastAPI routes**: `GET /forecast/{store_id}`, `POST /forecast/recommendations/{id}/decision` — same router pattern as your existing `batches` / `stores` routers from Phase 1.
- **No new infrastructure.** Postgres, Redis, Celery, FastAPI are all already provisioned — this volume is additive within the existing Docker Compose topology, not a new deployment target.

5.3 Request flow (text diagram) — the forecasting loop specifically

```
sales_history (accumulated daily from batch outcomes + POS-style CSV import)
        |
        v
[Celery beat: weekly forecast job]
        |
        v
[Forecasting model] <-- season/weather/event calendar context
        |  (predicted demand, recommended reorder qty, confidence)
        v
[reorder_recommendations table]
        |
        v
[Retailer Dashboard: Demand & Reorder tab] --> manager accepts/adjusts
        |
        v
(manager places real-world order -- outside InvenSight's scope)
        |
        v
new stock arrives --> ingested via existing Phase 1 CSV/manual entry --> becomes a `batches` row
        |
        v
[Volume I's markdown + claim + redistribution loop takes over]
```

Chapter 6 — Technology stack (extended)

6.1 What's added, and why it doesn't change the stack's shape

Layer	Volume I choice	This volume	Rationale
Forecasting model	—	**Scikit-Learn** (`GradientBoostingRegressor` or `HistGradientBoostingRegressor`) — same library already in your stack for markdown Model A	No new dependency; your team already knows this API from Phase 2
Time-series handling	—	Simple lag/rolling features engineered by hand (see Chapter 12), **not** a dedicated time-series library (Prophet, statsmodels)	Volume I made this exact call for the pre-alert stretch model (Ch. 11.3) — same reasoning applies: a well-featured gradient-boosted model is enough for a hackathon window, and avoids a new library's learning curve
Scheduling	Celery beat (existing)	Reused, second scheduled job	No new infra
Dashboard charting	—	A charting library already available in your frontend toolchain (e.g. Recharts)	Matches what Phase 6's Eco-Impact dashboard already needs
New APIs	—	Optional: a public holiday/local-events calendar API (already flagged as a Volume I stretch idea, Ch. 9.2) — now has a concrete consumer: it's a real input feature for the forecast, not just a pricing nicety	Reuse over new integration

The deliberate takeaway for this chapter: this addition requires zero new infrastructure decisions. Every new component is an instance of a pattern you already built in Phase 1/2.

Chapter 7 — Workflows: customer, retailer, and developer (extended)

7.1 Customer workflow

Unchanged from Volume I. This is a load-bearing point for the pitch: the forecasting layer is entirely invisible to the end consumer. Nothing about the SMS alert, claim page, or checkout changes.

7.2 Retailer / store staff workflow (extended)

- *(existing, Phase 6)* Reviews at-risk batches, sees suggested markdown price, accepts or overrides.
- **(new)** Weekly, opens the Demand & Reorder tab: reviews per-SKU recommendations, sees the trend chart and the "why" behind each number, accepts/adjusts/dismisses.
- **(new)** Places the actual order through their existing supplier process (outside InvenSight's scope — the hackathon build recommends, it does not procure).
- *(existing)* When new stock arrives, ingests it via the same CSV/manual/OCR path from Phase 1 — no new onboarding step for the retailer to learn.

7.3 Developer workflow

Unchanged in structure, extended in scope — same RULE.md discipline (phase branches, explicit permission gates, STATUS.md/DECISIONS.md updates) applies to the new phases in Chapter 15. See the companion `RULE.md` for the full rule set, now explicitly scoped to cover both volumes of work under one discipline.

Chapter 8 — Implementation plan: the short-timeline reality

8.1 The core constraint, stated plainly

You are not starting from zero — you have a working Phase 0/1 build (confirmed: containers healthy, schema live, seed data loaded). That is a real head start most teams attempting a “dual-sided platform” pitch will not have. The honest time budget for this volume’s additions, assuming you’re picking this up with limited hours left in your event:

- **If you have 8-10 hours left**: build Phases 8-9 only (forecasting data + model), skip the dedicated dashboard tab, and show the forecast output via the existing `/docs` API playground or a simple table — still a legitimate, demoable “we predict reorder quantities” story.
- **If you have 12-16 hours left**: Phases 8-11 (adds the real dashboard tab).
- **If you have the fuller runway (18h+)**: Phases 8-13, full dual-sided demo with both narratives rehearsed.

8.2 The one rule that matters most under time pressure

Do not touch the working Phase 0-7 code to “clean it up” or “make it fit better” with the new feature. The forecasting layer is additive by design specifically so you never need to. Any temptation to refactor the existing markdown model, the batches schema, or the claim flow to “integrate more elegantly” with forecasting is scope creep that risks the working system you already have. New tables, new routes, new routers — never edited files in the existing Phase 1-7 codebase except the one line needed to add a new dashboard tab/route registration.

Chapter 9 — Innovative angles & helpful third-party APIs (this volume's additions)

9.1 New APIs this volume actually needs

Need	Suggested API/tool	Why
Local events/holiday calendar	A public holiday calendar API (already flagged in Volume I Ch. 9.2 as a nice-to-have; now a real forecast input)	Demand spikes around festivals/holidays are exactly what a naive rolling-average forecast misses — this is a cheap, high-value feature
Weather (forecast window)	Reuses Volume I's OpenWeatherMap/Tomorrow.io integration (Ch. 6.2) — same key, same API, now consumed by two models instead of one	No new integration cost

9.2 Innovative angles specific to the forecasting pivot

- **"Why" explanations, not just numbers.** A bare "order 40 units" recommendation is far less trustworthy to a judge or a retailer than "order 40 units — trending +12% over 4 weeks, local festival in 6 days historically drives dairy demand up." Gradient-boosted models support feature-importance extraction cheaply (e.g. built-in `feature_importances_`, or a lightweight SHAP pass if time allows) — surface the top 2-3 factors per recommendation. This single feature does more for pitch credibility than any accuracy improvement would.
- **Forecast-vs-actual overlay as the single best demo visual for this half of the pitch** — a simple line chart with predicted vs. actual sales, updating as "time" advances through your seeded demo data, is the forecasting equivalent of Volume I's live-repricing "wow moment" (Ch. 15.1).
- **Cross-model connection as a talking point, not just an architecture note**: explicitly call out in the pitch that an under-performing-vs-forecast batch automatically gets flagged for earlier markdown — this is the single sentence that proves the "closed loop" claim isn't just a diagram, it's a real data dependency.

Chapter 10 — The machine learning model(s) (Model D added)

10.1 Models A, B, C — unchanged

Reuse exactly as specified in Volume I Chapter 11: Model A (markdown/price optimization), Model B (segment/channel classifier), Model C (predictive pre-alert, stretch).

10.2 Model D — Demand Forecasting (new)

Purpose: predict expected demand (units) for a given SKU at a given store over the next order cycle (e.g. next 7 days), and translate that into a recommended reorder quantity.

Approach: a regression model (`GradientBoostingRegressor` / `HistGradientBoostingRegressor` — same class as Model A, different target and features) predicting `units_sold_next_7d` from historical and contextual features (Chapter 12). The reorder quantity recommendation is then:

```
recommended_reorder_qty = predicted_demand_next_cycle
                          + safety_stock_buffer(category, forecast_confidence)
                          - current_qty_on_hand
```

where `safety_stock_buffer` is a simple, explainable rule (e.g. a percentage of predicted demand that scales inversely with the model's confidence for that SKU) — not itself a learned parameter for a hackathon build. Keeping this rule transparent is a deliberate choice: a judge asking "how did you get from a prediction to a number the retailer sees" should get an answer they can follow on a whiteboard, not a black box.

10.3 Confidence / uncertainty — do not skip this

A forecast without a stated confidence level is far weaker to present than one with it — this is your most important credibility move in Chapter 9's "why" framing. Cheapest hackathon-viable approach: train the same gradient-boosted regressor at multiple quantiles (e.g. 10th/50th/90th percentile via `loss="quantile"` in `GradientBoostingRegressor`), and use the spread between the low and high quantile predictions as your confidence band. This is a well-established, low-implementation-cost technique — not a research project.

10.4 Cold start — same principle as Model A, applied here

New stores/SKUs with insufficient history fall back to a category-level average demand curve (directly reusing the same cold-start lookup-table pattern from Volume I Ch. 11.2), visibly flagged on the dashboard as "estimated from category averages" rather than "predicted for this store." Consistency with the existing cold-start UX pattern is itself worth pointing out in the pitch — it shows a designed system, not two bolted-together features with different behaviors.

Chapter 11 — Data requirements, in detail (this volume's additions)

11.1 New table: `sales_history`

Field	Notes
store_id, sku	Same identifiers as `batches` — this is the join key back into the existing schema
date	One row per SKU per store per day
units_sold	The core target signal
price_at_time	For reconstructing price-elasticity effects, same rationale as Volume I Ch. 12.2
promotion_flag	Whether a markdown was active that day — critical, since a markdown-driven sale spike is not "organic demand" and must be distinguishable in training
stockout_flag	Whether the SKU was unavailable that day — without this, missing sales get misread as "no demand" rather than "no stock to sell"

11.2 New table: `reorder_recommendations`

Field	Notes
store_id, sku, forecast_run_date	Identifies which forecast cycle this belongs to
predicted_demand, confidence_low, confidence_high	Model D's output
recommended_qty	The translated reorder number (see Ch. 10.2)
top_factors	Short text/JSON of the top contributing features, for the "why" UI (Ch. 9.2)
retailer_decision	accepted / adjusted / dismissed, and the adjusted value if applicable — this becomes a valuable feedback signal for future model iterations

11.3 Context data (mostly reused from Volume I)

- Weather — reused integration (Ch. 6.1 of this volume)
- Local events/holiday calendar — new integration (Ch. 9.1)
- Existing `batches` outcome data (sold/expired/donated) — already collected by Phase 1-5, now double-purposed as a forecasting feature source, not just a markdown-retrain source

11.4 Handling the lack of real historical data — same honest approach as Volume I

Generate a synthetic `sales_history` dataset with realistic per-category seasonality and trend patterns plus noise, seeded for reproducibility — directly following the same pattern and same honesty commitment as Volume I Ch. 13.5 (state to judges that it's synthetic, document the generation logic so a real POS feed can be swapped in later without changing the model interface).

Chapter 12 — Parameters, training/testing strategy & algorithms (Model D)

12.1 Feature list

Feature	Type	Notes
units_sold_lag_7d, lag_14d, lag_28d	Numeric	Recent sales history at increasing lookback windows
rolling_avg_7d, rolling_avg_28d	Numeric	Smoothed trend signal
day_of_week, is_weekend, is_holiday	Boolean/Numeric	Same temporal features as Model A
category	Categorical	One-hot or target-encoded
temperature_forecast, precip_probability	Numeric	Reused weather integration
local_event_flag	Boolean	New calendar integration (Ch. 9.1)
promotion_planned_flag	Boolean	Whether a markdown is scheduled in the forecast window — demand under a planned promotion is meaningfully different from baseline
stockout_rate_recent	Numeric	Corrects for demand-suppression bias from recent stockouts (Ch. 11.1)

12.2 Target variable

`units_sold_next_7d` — a direct regression target, mirroring the structure of Model A's target (Volume I Ch. 13.2) but predicting forward in time rather than at-a-price.

12.3 Train/test split — same non-negotiable rule as Volume I

Time-based split, never random — train on earlier weeks, validate on the most recent. This matters even more for Model D than Model A, since demand forecasting is inherently a time-series problem; a random split here would be a more serious methodological error than in the markdown case, and worth specifically flagging as something you got right if a judge probes your validation approach.

12.4 Algorithms, by stage

Stage	Algorithm	Why
Baseline	Simple moving average / naive "same as last week"	Cheap sanity-check floor — if your model can't beat this, something is wrong before you invest further
Production candidate	Gradient Boosted Trees, quantile loss for confidence bands (Ch. 10.3)	Same library/class as Model A — no new tooling to learn under time pressure
Evaluation	MAE / RMSE against actual `units_sold`, plus a simple "beat the naive baseline by X%" framing for the pitch	The baseline comparison is the single most judge-legible accuracy statement you can make — lead with it over a raw MAE number

12.5 Handling the honesty requirement — restated because it matters twice as much here

Because this model is trained on synthetic data (Ch. 11.4) *and* is inherently harder to validate in a live hackathon demo than the markdown model (you can't wait a week to prove a forecast was right), be explicit and proactive about this in the pitch, rather than waiting for a judge to ask. Frame it as: **"the pipeline is real and swappable;**

the specific numbers you're seeing are demonstrating the mechanism, not a production-accuracy claim." This preempts the single most likely tough question in Q&A for this half of the project.

Chapter 13 — The ecosystem: integration & pitch framing (updated)

13.1 The stakeholder ecosystem — one stakeholder added

Volume I's ecosystem (Ch. 14.1) gets one addition: **suppliers/distributors**, as a future integration point — once a retailer accepts a reorder recommendation, that recommendation could eventually be pushed as a draft purchase order to a supplier's own ordering system. This is explicitly **out of scope for the hackathon build** — mention it only as a stated future direction in the ecosystem slide, not something to attempt building.

13.2 Integration path over time, updated

- **MVP (hackathon)**: forecast recommendations are advisory-only, reviewed and manually acted on by the retailer — no automated ordering.
- **Growth**: recommendation acceptance/adjustment data (Ch. 11.2's `retailer_decision` field) becomes training signal for a better-calibrated model — a natural, low-effort feedback loop already captured by the schema.
- **Maturity**: the supplier-integration extension above, plus the same municipal/ESG reporting expansion already described in Volume I Ch. 14.2, now enriched with "waste avoided through better ordering" as an additional reportable metric alongside "waste rescued through markdown."

13.3 Pitch structure, updated for the dual narrative

```
Problem (both halves, Ch. 1) → Solution (Ch. 2, the closed loop) → Live demo (Ch. 14, both halves)
→ Differentiation (Ch. 3, now vs. two competitor categories) → Business model (unchanged from Volume I)
→ Impact metrics (Eco-Impact dashboard + new forecast-accuracy-vs-baseline stat)
→ The ask
```

The single most important structural point for the pitch script: **open with the loop diagram (Ch. 4.1/4.2), not with either half in isolation.** Teams that pitch forecasting and liquidation as two separate features read as two ideas awkwardly combined; teams that open with the loop read as one coherent system. Spend the first 30 seconds of the pitch on that diagram before touching either demo.

Chapter 14 — Demo plan (extended run-of-show)

14.1 Suggested extended flow — building on Volume I's 3.5-4 minute script

Volume I's demo (Ch. 15.1) starts at "store dashboard, dairy batch nearing expiry." This volume's addition goes **before** that, not instead of it:

- **0:00-0:30** — Open on the loop diagram (Ch. 4.2), stated as one sentence: *"InvenSight predicts what to order, watches what's at risk, and rescues what would otherwise be lost — one system, one data flow."*
- **0:30-1:00** — Demand & Reorder tab: show the forecast-vs-actual chart for a seeded SKU, and the "why" behind one recommendation (Ch. 9.2). Accept the recommendation on screen.
- **1:00-1:15** — **The connective-tissue beat, the most important 15 seconds of the added demo**: point out a *different* SKU that under-performed its forecast, and show it already flagged for earlier markdown on the existing dashboard — proving the loop is real, not just a diagram.
- **1:15 onward** — Hand off directly into Volume I's existing run-of-show (Ch. 15.1, steps starting at their 0:30 mark) — the dairy batch, the live repricing, the SMS alert, the claim page, the NGO queue, the Eco-Impact close.

Total extended demo: roughly 5-5.5 minutes. If your event's time slot is shorter, cut the 0:00-1:15 addition down to just the loop-diagram sentence and the "why" chart — the connective-tissue beat is worth protecting over the chart if you must choose one.

14.2 Risk mitigation — same discipline as Volume I, one addition

Everything in Volume I Ch. 15.2 applies unchanged (pre-recorded backup, seeded reproducible data, rehearsed payment-failure answer). Add one more rehearsed answer specific to this volume: **"how do you validate a forecast when you can't wait a week during a demo?"** — the honest answer is in Chapter 12.5; rehearse saying it confidently rather than defensively.

Chapter 15 — Implementation plan, phase by phase, continuing from Phase 7

15.1 Where you actually are right now

Phases 0 and 1 are confirmed built and verified (containers healthy, schema live, seed data loaded, endpoints functional). Phases 2-7 from Volume I remain the plan for the liquidation/redistribution half of the product — this chapter does not repeat them; it adds the phases needed for the forecasting half, numbered to continue the same sequence so both volumes read as one continuous roadmap in your repo's `STATUS.md`.

15.2 New phases

Phase	Duration (rough)	Deliverable	Cut-scope fallback
8 — Sales history & synthetic data	3h	`sales_history` table live, synthetic generator script (mirrors Phase 0's `seed.py` pattern), seeded with realistic per-category seasonality	Smaller synthetic dataset (fewer SKUs/weeks) — still demoable, just thinner trend lines
9 — Demand forecasting model (Model D)	4h	Baseline (naive average) working first, then the gradient-boosted quantile model behind the same swappable interface pattern used for Model A (Volume I Ch. 8.1, Phase 2)	Ship the naive baseline alone if the gradient-boosted model isn't converging — still a legitimate "we forecast demand" demo, framed honestly as v1
10 — Reorder recommendation logic	2h	`reorder_recommendations` table populated, the qty/confidence/top-factors computation from Ch. 10.2-10.3 wired up, `GET /forecast/{store_id}` endpoint live	Skip the safety-stock buffer nuance — return raw predicted demand as the recommendation if time-constrained
11 — Demand & Reorder dashboard tab	4h	New React tab: trend chart, recommendation table, accept/adjust/dismiss controls, "why" display	If time-constrained, skip the accept/adjust UI and just render the table read-only — still shows the core value, loses the feedback-loop story
12 — Loop integration & validation	2h	The specific connective-tissue behavior from Ch. 14.1 (under-forecast SKU triggers earlier markdown flag) actually implemented and testable, not just narrated	This is the highest-priority item to protect if time runs short elsewhere — it's the single feature that proves the "closed loop" pitch claim is real
13 — Extended demo rehearsal	2h	Full run-through of Ch. 14.1's extended flow, backup recording, the Ch. 14.2 Q&A answer rehearsed	—

Added total: ~17 hours, on top of Volume I's ~32-hour budget for Phases 0-7. If your remaining time is short, Chapter 8.1's compressed-timeline guidance (Phases 8-9 only, or 8-11) tells you where to stop.

15.3 Priority order if time is very short

If you only have a few hours left before the deadline: **Phase 8 → Phase 9 (baseline only, skip quantile) →**

Phase 12. In that order, you can genuinely say in the pitch “we forecast demand from historical data, and an under-performing item automatically gets flagged for markdown” — which is the entire structural claim of this volume — without having built the dashboard UI at all. The UI (Phase 11) is the most cuttable phase relative to how much pitch-narrative weight it carries; the loop-integration behavior (Phase 12) is the least cuttable, because it’s the one thing a competitor’s forecasting-only or markdown-only product structurally cannot claim.

Chapter 16 — RULE.md

See the companion `RULE.md` file, provided separately alongside this document. It governs both this volume's new phases (8-13) and remains consistent with the working discipline already in place for Phases 0-7 — one rule set for the whole project, not a separate one per volume.

Chapter 17 — Authenticity, sourcing, and the 24-hour execution plan

This chapter exists because the project's real starting point turned out to be different from what Volume II originally assumed, and because a hackathon submission has to be honest and checkable, not just technically good. Everything below reflects the actual, current state and the actual plan for the 24-hour window.

17.1 The honest starting inventory

Before any hackathon-window work begins, the actual pre-existing state is:

Asset	Status	What this means for the 24 hours
Dashboard UI (layout, components)	Partial, built before the event	Reusable as a starting point — extend, don't rebuild the shell
ML model (markdown/pricing)	Prototype, worked inconsistently, wrong objective for this problem statement	**Not reused.** Requires full retraining against a different target, so treat it as new work, not carried-over work
Core concept (waste reduction, expiration tracking, dual-sided value)	Idea only, no additional code beyond the above	Ideation is not "prior work" in the sense a judge would weigh — safe to state as the origin of the vision without it counting against the 24-hour build

The practical consequence: **almost everything of technical substance is genuinely built within the 24-hour window.** This is a stronger, simpler position than trying to claim credit for a working backend — there is no working backend from before that a tough technical question could expose gaps in.

17.2 Repository migration — doing this before any hackathon-window work starts

1. Create the new repository (e.g. `invensight`) on GitHub — empty, no auto-generated README/`.gitignore`/license, so nothing conflicts with the imported history.
2. Add collaborators immediately.
3. Clone the existing project as the base, and re-point it at the new remote:

   ```
   git clone <existing-repo-url> invensight
   cd invensight
   git remote rename origin old-origin
   git remote add origin <new-repo-url>
   git push -u origin --all
   ```

   This step matters specifically because it carries over **every real commit with its real, original timestamp** — the project's actual history stays checkable rather than being hidden or rewritten. This is the difference between an honest continuation and something a judge would be right to be suspicious of.
4. Confirm all branches and commit dates landed correctly on the new repo's GitHub page.

17.3 Marking the honest starting line, in the open

Create a `hackathon-2026-start` branch and a root-level `HACKATHON-CONTEXT.md` stating plainly what predates the event (per the table in 17.1) and what the hackathon window covers from that commit forward. This file is not a formality — it is the answer to the single most likely tough question a judge asks, delivered before they have to ask it, in the team's own words rather than inferred from a suspicious-looking repo.

17.4 Ongoing source citation — a standing practice, not a one-time disclosure

Two documentation habits run for the entire 24 hours, not just at the start:

- `SOURCES.md` — a running table logging every reused code pattern, design element, or third-party approach as it is used, with what it is and where it came from. Anything genuinely built fresh during the window gets no entry.
- `DECISIONS.md`**, extended** — every entry now includes a source field: “original to this hackathon” or “adapted from `<specific source>`.”

This is added to `RULE.md` as a fifth standing rule (see the companion file) — citation is treated as part of the working discipline, at the same priority as branch-per-phase and permission-before-push, not an afterthought bolted on before submission.

17.5 AI-assistance disclosure — a separate check from code-reuse disclosure

Where AI tools (including this playbook and any AI-assisted code) were used in the build, check the specific hackathon’s disclosure requirements and prepare a short, accurate statement of what was AI-assisted vs. team-authored. This is commonly a separate rule from general code-reuse policy and is easy to miss if only the reuse question is asked about.

17.6 The revised 24-hour plan, reflecting the honest starting point

Because the ML model is effectively new work (17.1), the phase plan from Chapter 15 is re-weighted for a single 24-hour window, prioritizing the pieces that are genuinely new over the one piece (dashboard shell) that has a head start:

Hours	Focus	Corresponds to
0-1	Repo migration (17.2-17.3), confirm the reused dashboard shell still runs cleanly, `SOURCES.md` / `DECISIONS.md` initialized	Setup
1-5	Core inventory/batch data model and synthetic sales history generation	Phase 8
5-11	Demand forecasting model, built fresh — baseline first, gradient-boosted quantile model second	Phase 9
11-15	Reorder recommendation logic, wired into the reused dashboard shell	Phases 10-11
15-19	Markdown/liquidation loop connection — the single most important differentiator to protect if time runs short	Phase 12
19-21	Eco-impact / reorder recommendation visuals, the demo’s key visual moment	Chapter 14
21-23	Pitch rehearsal, backup recording, `SOURCES.md` / `DECISIONS.md` final pass for accuracy before submission	Phase 13
23-24	Buffer	—

The one priority rule that overrides all others under time pressure: if hours run short, the loop-connection behavior (hour 15-19 block) is protected above dashboard polish or visual extras — it is the one feature a forecasting-only or liquidation-only competitor structurally cannot claim, and it is what makes the “always had this vision” pitch line true rather than aspirational.

17.7 The pitch line, final version

> “The idea — reducing food waste by connecting expiration tracking to real inventory decisions — is something we’d been thinking about before today. But this problem statement is what actually pushed us to build the piece that made it work: demand forecasting. Almost everything you’re seeing — the model, the forecasting logic, the connection between ordering and markdown — was built in the last 24 hours, and we can walk

> through exactly what and where in our sources log if you’re curious.”

That last clause — offering the sources log proactively — is deliberate. It converts a potential gotcha into a demonstration of rigor.